



EVERYONE

Viva questions and answers

Every question we expect, who answers it, and a short correct answer. The owner's own brief has more detail.

How to answer

Answer in two or three sentences, then offer to show it: "I can open that file." Say what the code does, not what you wish it did. If a question is outside your layer, say "that is Chandra's part" and hand it over. If you do not know, say so and say where in the code you would look.

A. Questions that cross layers

Q What happens, step by step, when a student presses Apply?

KARAN STARTS, THEN TIYA, CHANDRA, YUVRAJ

1. The browser sends a POST to `/app/apply` with a hidden CSRF token. 2. `AuthFilter` checks there is a signed in session (and the officer role for `/admin/*`), and on a POST that the CSRF token matches. 3. `ApplicationServlet.doPost` loads the student and the drive and asks `DriveService.check` for the verdict. 4. If already applied or not eligible, it flashes a message and redirects. 5. Otherwise it calls `ApplicationService.apply`, a stateless EJB, inside a container managed transaction: it persists a `JobApplication` and queues a notification message. 6. Hibernate writes the row to MySQL, and the unique constraint on student and drive is the last guard. 7. The servlet redirects to `/app/applications`, which forwards to a JSP that lists the rows. 8. Meanwhile the message driven bean logs "Notification delivered".

Q Why do you redirect after a POST instead of showing the page directly? KARAN, TIYA

Post, Redirect, Get. If the browser were left on the POST result, pressing refresh would submit the form again and apply twice. A redirect makes the browser load a fresh GET, so refresh is harmless. The flash message survives the redirect because it is stored in the session and removed after it is shown once.

Q Why do you check eligibility both on the page and in the servlet? KARAN, CHANDRA

The page only hides the button. A page can be stale, a form can be replayed, and a request can be written by hand, so the rule check that matters is the one on the write path. The database adds a third guard, a unique constraint on student and drive.

Q You use both Hibernate and JDBC. Why? YUVRAJ, CHANDRA

Ordinary reads and writes are object shaped, so JPA with Hibernate is the right tool: one object, one row. Reports are the opposite: they aggregate across tables, and plain SQL says that more directly than mapped objects. Both use the same server managed connection pool by the same JNDI name, so there is one pool and one database.

Q Where is the database password kept? YUVRAJ

In `WEB-INF/glassfish-resources.xml`, which defines the connection pool and the datasource `java:app/jdbc/placementDS`. The application code asks for that name and never sees a URL or a password. The only exception is `MockScoreBean.addScore`, the classroom style bean, which has them written in the code on purpose, to show the difference.

Q How do the JSP pages get their data? KINJAL, KARAN

The servlet puts the data on the request as attributes, then forwards with `RequestDispatcher` to a JSP under `WEB-INF/views`. The JSP reads the attributes with EL and loops with JSTL. There is no Java in the page, because scriptlets are switched off in `web.xml`. A browser cannot ask for a JSP directly because files under `WEB-INF` are never served.

Q How is this different from the practical where we used a Stateful bean with DriverManager? YUVRAJ

The practical opened a connection with `DriverManager` with the URL and password in the code, joined the values into the SQL text, and never closed the connection. We keep that as `MockScoreBean.addScore` and run it live on the Classroom JDBC page. Beside it, `addScoreSafely` takes a connection from the pool, binds values with placeholders and closes everything. Both write to MySQL.

Q Which parts of the course syllabus did you use? KARAN

Servlets with lifecycle and annotations, `RequestDispatcher`, cookies, `HttpSession` and a listener, file upload and download, non blocking I/O, JSP with actions, EL and JSTL, EJB of every kind (stateless, stateful, singleton, message driven, interceptor), JNDI, JPA with Hibernate, and JDBC. The table in `00-start-here` shows where each one is.

Q What would you do next if you had more time? KARAN

A recruiter role with its own login and an approval step by the officer, real email delivery for notifications, password reset and email confirmation links, and paging on the long lists.

Q How did you test it? CHANDRA

146 automated JUnit tests covering the eligibility rules, validation, password hashing, email domain checks, uploads, and live database tests for the JDBC layer. They run with `.\test.ps1` and finish in a few seconds. The pages were also walked through in a browser.

B. Yuvraj: MySQL and JDBC

13 questions. The full brief is `02-yuvraj-mysql-jdbc`.

Q What happens if the password in the pool is wrong?

MySQL refuses the login with "Access denied for user placement_user". The password lives in one place, `glassfish-resources.xml` (line 52). The pool is set with `ping="true"`, so GlassFish tests it and reports the failure. In the app, `describeConnection()` catches the `SQLException` (lines 115 to 119) and the Reports card shows "Unavailable" with MySQL's message instead of crashing. Pages that go through Hibernate will also fail. Fix the password in the XML and redeploy with `.\run.ps1`. No Java file changes.

Q What is a connection pool?

A set of already open, already authenticated MySQL connections kept by the server. Code borrows one with `getConnection()` and gives it back with `close()`. Ours (lines 41 to 47): 4 connections opened to start, at most 24, idle ones dropped after 300 seconds, and each is validated before it is handed out.

Q Why not open a connection per request?

Opening a TCP connection and logging in to MySQL costs far more than the query itself, and MySQL allows only a limited number of connections. A pool pays that cost once and reuses. That is exactly what `MockScoreBean.addScore` does wrong: it opens a new connection on every call and never closes it.

Q Is this safe from SQL injection? Show me.

Yes for values. Show `searchCandidates` lines 271 to 299: the SQL only ever gets `?` added, and values are bound with `setObject`. Then run the mysql `PREPARE` demo: concatenated returns 26 rows, bound returns 0. Two honest limits. A table name cannot be a `?`, so `requireKnownTable` (explorer, line 347) accepts only names the database itself reported, and `countRows` (report DAO, 426 to 437) uses a fixed list. And the classroom method is unsafe on purpose, which is why the servlet validates first.

Q What goes wrong with the classroom version?

Four things, all visible in `addScore`. It opens a new connection on every call and never closes it, so connections leak. The URL and password are in the source. The SQL is built by joining strings, so a name like `O'Brien` ends the string early and the statement is rejected, which is SQL injection in its everyday form. And errors are only printed: the practical hides the failure completely, ours returns 0 so the page shows an error, but the exception itself still only goes to the server log. The pooled method saves the same name because the value is bound, not joined.

Q What is the difference between Statement and PreparedStatement?

A Statement sends the finished SQL text. A PreparedStatement sends the SQL with `?` placeholders, and the values travel separately, typed, so they can never be read as SQL. It also lets the database reuse the parsed statement. We still use a plain Statement in a few places where no user value is joined in: row counts over a fixed or database-reported table list, and the console, which runs officer typed reads behind the guard.

Q What if you forget to close the connection?

With a pool, `close()` does not close the socket, it returns the connection. Forget it and the connection is never returned. After 24 leaks the pool is empty and every request waits. That is why every connection here is in a try with resources block, and why `closeDriverAndRejectPending` closes in a `finally`.

Q Why JDBC for reports if you already have Hibernate?

Hibernate is right for loading an object, changing a field and saving it. Reports aggregate across tables and return numbers, never written back. Plain SQL with `COUNT`, `GROUP BY` and a LEFT JOIN is clearer than forcing it into JPQL. Both use the same datasource, so it is one pool, one set of credentials.

Q Explain the transaction in closeDriverAndRejectPending.

Two updates must succeed together: set the drive to CLOSED, and set its APPLIED applications to REJECTED. `setAutoCommit(false)`, run both, `commit()`. On any `SQLException`, `rollback()` undoes both. The finally block restores auto commit, then closes. The bean is `NOT_SUPPORTED` so the container does not start its own JTA transaction, which would conflict with one driven by hand.

Q Can someone destroy data through the SQL console?

Three layers. One: the guard (must start with select, with, show, describe or explain, one statement, no forbidden keyword such as drop, update or into). Two: `setReadOnly(true)` on the connection, so the server refuses writes even if the guard were fooled. Three: the database user only has rights on `placement_cell`. Be honest about layer three: that user does have all privileges on this schema, so layers one and two are what stop writes here. Rows are capped at 200 and a query is cancelled after 5 seconds.

Q Why is the MySQL driver not inside the WAR? Do you still need Class.forName?

The pool is created by GlassFish before the application loads, so the driver must be on the server's classpath: `run.ps1` copies it into `domain1\lib`. And `Class.forName` is not needed with a modern JDBC 4 driver, which registers itself. It appears in `MockScoreBean.addScore` only because the practical uses it.

Q Where is DatabaseMetaData used?

Two places. `describeConnection()` (report DAO, 100 to 121) fills the "live connection" card: product name and version, driver, URL, user, isolation. `describe()` (explorer DAO, 99 to 170) reads columns, primary keys, foreign keys and indexes for the Structure tab. `tableNames()` uses `getTables` as the whitelist.

Q The password is in plain text in an XML file. Is that acceptable?

It is a local database made for this project and the user only has rights on one schema. It is listed as an accepted risk in `docs/SECURITY.md` (item A1). The production step would be a GlassFish password alias or a secret from the environment. The point of the design is that it is in one server file, not compiled into classes.

C. Tiya: Accounts and security

13 questions. The full brief is `03-tiya-auth-security`.

Q How is an account created?

Student self sign up only, at `/register`. `RegisterServlet` checks the CSRF token, name, email (four checks), password strength, roll number, branch, batch year, CGPA, backlogs and phone. Only if there are no errors does it call `AuthService.registerStudent`, which checks for a duplicate email or roll number again, hashes the password, and saves an `app_user` row plus a student profile. The role is hard coded to STUDENT. Officers exist only through the seeder, never through sign up.

Q Would it accept a fake email?

It rejects an address whose domain does not exist: `test@thisdomaindoesnotexist123.com` gets "That domain doesn't appear to accept email". It rejects a malformed one such as `admin@campus`, and 43 known disposable providers such as mailinator. It would still accept an invented mailbox at a real domain, for example `asdf1234@gmail.com`, because we do not check the mailbox.

Q So do you verify the email really exists?

Done: shape check (`Validators.isEmail`), disposable blocklist, and a DNS lookup for an MX record with A and AAAA as fallback (`EmailDomainVerifier`). That proves the domain can receive mail. **Not done:** no confirmation link, no OTP, no test send, and nothing proves the person owns the mailbox. That needs an SMTP account we do not have. It is the recorded next step (Security review S10). If DNS cannot be reached at all, the check lets the sign up through and logs a warning, on purpose.

Q Does it hide the password?

Yes on screen: the inputs are `type="password"` on both forms. The forms use POST, so the password is in the body, not the URL. When a form fails, the servlet forwards the page back with no password value, so the boxes come back empty. The audit trail records the call as `authenticate(email, [redacted])`. Honest limit: the app runs on plain HTTP on localhost, so on a real network it would need HTTPS.

Q How is the password stored? Can you see it?

As `pbkdf2$120000$salt$hash` in `app_user.password_hash`: PBKDF2 with HMAC SHA 256, 120,000 rounds, a 16 byte random salt per account, 256 bit output (`PasswordHasher`). It is one way, so we cannot read the original. Login hashes the typed password with the stored salt and compares in constant time. The salt means two people with the same password get different hashes.

Q What happens after wrong passwords?

Each wrong password adds one to `failed_attempts` in the database (`AuthService` lines 74 to 85). The message is "Email or password is incorrect." with HTTP 401. With two or one attempts left it adds a countdown. The fifth sets `locked_until` fifteen minutes ahead. While locked, even the correct password is refused. A correct sign in resets the counter. The officer can clear a lockout early on the student's page. The lock lives in the database, so restarting the server does not clear it, and it counts per account, not per computer.

Q Can someone tell which emails have accounts?

Not from the wrong password message: unknown email and wrong password read the same, and an unknown email still burns one hash so the timing matches. Two small leaks exist and you should say so: the countdown and lockout messages only appear for real accounts, and the register page says "An account already exists for that address".

Q Can a student open the admin pages?

No. `AuthFilter` line 63 checks the role for every path under `/admin`, and a student gets HTTP 403 with the "Not allowed" page. Someone not signed in is redirected to login. The path it tests is cleaned first by `RequestPaths.canonical`, so `//admin/database` and `/admin;x=1/students` are refused too. We tested all of these as a student. This was a real bug in an earlier version: the check read the raw URL, the container routed on the cleaned one, and a doubled slash got through. It is fixed and covered by `RequestPathsTest`.

Q What is session fixation and how do you stop it?

An attacker plants a known session id in the victim's browser, waits for the victim to sign in, then uses the same id. We call `request.changeSessionId()` right after a successful password check (`LoginServlet` line 98), so the pre login id becomes useless. Only cookies are allowed for the id (`web.xml` line 30), so it cannot sit in a URL.

Q How long does a session last, and who counts them?

30 minutes of inactivity: `web.xml` line 23, and `SessionTracker` line 24 sets the same 30 minutes when each session is created. Logout invalidates it at once. `SessionTracker` implements `HttpSessionListener`; the container calls it on creation and on invalidation or timeout, and the officer's System page shows "Sessions open now".

Q Which cookies do you set, and with what flags?

`JSESSIONID`: `HttpOnly` (in `web.xml`). `remember_email` (only if the box is ticked) and `last_visit`: `HttpOnly`, 30 days, path `/placement`, values percent encoded. `portal_theme`: one year, not `HttpOnly`, only says light or dark. No cookie holds a password. Not set: the `Secure` flag, because a secure cookie would never be sent over plain HTTP on localhost (accepted risk A2 in the review), and no `SameSite` attribute is set in our code.

Q Why are the JSPs under WEB-INF? Forward or redirect?

Anything under `WEB-INF` cannot be requested by URL: we tested `/placement/WEB-INF/views/login.jsp` and got 404. Only a servlet can reach a view, by `RequestDispatcher.forward`, which happens inside the server: the address bar keeps the servlet URL. A redirect sends the browser a 302 and it makes a second request. We forward to show a page, including a failed login, and redirect after every successful POST (Post-Redirect-Get) so refresh cannot repeat the action.

Q What are CSRF and the Content Security Policy?

CSRF: another site making your browser submit a form to us. Every POST needs the random per session token in a hidden field (`Web.csrfValid`). **CSP:** `script-src 'self'` with no inline scripts, so the browser refuses any script we did not serve. Plus `frame-ancestors 'none'` and `X-Frame-Options: DENY` against clickjacking, `form-action 'self'`, `nosniff`, and no-store on every page that is not a static asset. Not set: HSTS, because we have no HTTPS.

D. Kinjal: Views, validation and uploads

14 questions. The full brief is `04-kinjal-views-validation`.

Q Why can't I open your JSPs directly?

They live under `WEB-INF/views`, and the container never serves anything under WEB-INF to a browser. The only way in is `RequestDispatcher.forward` from a servlet, which is what `Web.render` does. Try it: `/placement/WEB-INF/views/drives.jsp` gives a 404. The one exception is `index.jsp`, the welcome file, and it holds no markup.

Q Why are scriptlets disabled, and where?

In `web.xml`, `<scripting-invalid>true</scripting-invalid>` for `*.jsp` and `*.jspx`. Java inside a page mixes logic into the view, cannot be tested, and is where injection bugs hide. With it off, a page containing a scriptlet fails to compile, so the rule cannot be broken by accident.

Q Include directive or `jsp:include` ? Which do you use?

The include directive, `<%@ include file="layout-top.jspf" %>`. It is a static include: the fragment is pasted in when the page is translated, so the whole page becomes one servlet and variables like `ctx` and `me` set in the fragment are visible to the page. `jsp:include` would run at request time as a separate resource. We do not use it.

Q What does `c:out` do, and why is it on every user value?

It escapes `& < > " '` before printing. Without it, a name like `"><script>alert(1)</script>` would be written into the page as live markup. With it, the register page sends back `"><script>alert(1)</script>`, which the browser shows as plain text. Plain EL like `${x}` does not escape. I checked this against the running register page.

It is used inside attributes too, not only in page text. The `data-confirm` messages that contain a name (`admin-companies.jsp` line 52, `admin-student-detail.jsp` lines 101 and 129, `my-applications.jsp` line 52) use `<c:out value='${co.name}' />` inside the attribute. Plain EL remains only for numbers and server made values such as ids, the csrf token and status labels. One client supplied exception: the theme cookie value on the html tag, which only the person's own browser can set.

Q How does a servlet hand data to a JSP?

It calls `request.setAttribute("drives", drives)` and then forwards. The JSP reads `${drives}`. EL searches page, request, session, then application scope for the name. The address bar keeps the servlet URL because a forward happens inside the server.

Q Forward or redirect, and when?

Forward to show a page: same request, attributes survive, no second round trip. Redirect after a successful POST so a refresh does not repeat the write. A message that must survive the redirect goes in the session as `flashSuccess` or `flashError`, and `layout-top.jspf` removes it with `c:remove` after printing.

Q Is validation done in the browser or on the server?

Both, but only the server counts. Inputs carry `required`, `min`, `max`, `maxlength`, `type="email"`, and the forms also carry `novalidate`, so the browser does not block a submit or show its own bubbles. `Validators` does the real check and the page shows the message under the field with `aria-invalid="true"`. The register page says it plainly: "Every field is checked again on the server before anything is saved." Anyone can bypass a browser with curl.

Q Why is there no inline JavaScript?

The Content-Security-Policy header says `script-src 'self'` with no `'unsafe-inline'`, so the browser runs only scripts served from our own origin. An injected script tag would sit there unused. That only works if no page has a script block or an onclick, so everything is in `app.js`. Confirm dialogs use a `data-confirm` attribute and one listener. Inline *styles* are allowed by the policy and a few pages use them.

Q How does light and dark mode work?

Clicking the icon calls `/theme?mode=dark`. `ThemeServlet` stores `portal_theme` in a one year cookie, accepting only `light` or `dark`. Each page reads it with `cookie.portal_theme.value` and puts it on the html tag as `data-theme`. `app.css` line 102 swaps the colour tokens under `[data-theme="dark"]`. It does not follow the operating system setting.

Q How is the resume upload kept safe?

Four checks and one placement rule (plus a friendly message for an oversize file). `@MultipartConfig` caps a file at 2 MB and a request at 3 MB. The extension must be pdf, doc or docx. The first bytes must match that format (`%PDF` , or the zip or OLE signature). An oversize file is caught and answered with a normal message (`ProfileServlet` lines 73 to 82, `AuthFilter` lines 70 to 80). The saved name is generated (`resume-1-a1b2c3d4.pdf`), so the browser's file name is never used as a path. The file goes to `domain1/placement-resumes` , outside the deployed app, so a redeploy does not wipe it.

Q How is a resume served back, and can a student read someone else's?

Through `/app/resume` , not a static link. The servlet finds the file name from the student record, resolves it inside the upload folder only, and sends it with `Content-Disposition: attachment` and `nosniff` . A student asking for another student's id gets 403. The officer may fetch any.

Q Explain the non blocking bulk import in plain words.

Normally `read()` makes a server thread wait for the next byte, so a slow upload ties up a thread doing nothing. In `BulkImportServlet` the request is put in async mode (line 67) and a `ReadListener` is attached (line 73). The container calls `onDataAvailable` only when bytes are ready, we read while `isReady()` is true, then the thread goes back to the pool. When the body ends, `onAllDataRead` validates the rows, writes JSON and calls `async.complete()` . Limits: 512 KB and a 30 second timeout. The non blocking part is reading the request. The database work for each row still runs on that callback thread.

Q Is the import a file upload?

No. The officer pastes text into a box and `app.js` sends it with `fetch` as `text/csv` . It is not multipart, which is why it is a different mechanism from the resume upload. The CSRF token goes in the query string because there is no form body to carry it. The script writes the JSON reply into the page with `textContent` , never `innerHTML` .

Q What accessibility work did you do?

What is in the markup: `lang="en"` on every page; a `label for` on form fields; `aria-invalid` set by the server with visible error text, so colour is not the only signal; `aria-hidden` on the logo and icon sprite; `aria-label="Menu"` on the menu toggle; a visible `:focus-visible` outline, including on the CSS only menus; transitions only when the user has not asked for reduced motion; and a written contrast audit in the comments of `app.css` . Not done: no skip link, and the theme and sign out icons have `title` text but no `aria-label` .

E. Chandra: Entities and EJB

10 questions. The full brief is `05-chandra-entities-ejb` .

Q Stateless versus stateful session bean?

Stateless keeps nothing between calls, so any pooled instance serves any caller (the three services). Stateful is one instance per client that remembers its fields (`ShortlistBean`).

Q Why is the shortlist bean stateful, and where is it held?

The officer ticks names across page loads, so the selection belongs to one person. The instance lives in the EJB container (30 minute timeout), the reference in that officer's HttpSession. A servlet field would be shared by all officers.

Q Singleton bean and @Startup? Interceptor?

A singleton is one instance for the whole app; `@Startup` creates it at deploy. An interceptor's `@AroundInvoke` method wraps a business method and calls `ctx.proceed()` , so the beans hold no audit code.

Q What is JNDI, and how does @EJB differ from a lookup?

JNDI is the container's name service: look up a name, get a resource, no URL or password in code. `@EJB` looks up once at creation; an explicit lookup gives a new stateful instance each time.

Q What does @PersistenceContext do?

It injects a container managed, transaction scoped `EntityManager` for `placementPU` . You never open or commit it.

Q Why Hibernate? JPA versus Hibernate?

JPA is the specification, Hibernate the implementation, named in `persistence.xml` . It writes the SQL, so only the dialect changes for another database.

Q Which side owns the foreign key, and how is a double application stopped?

The side with `@JoinColumn` , for example `JobApplication.student` . `hasApplied` checks first, and the unique key `uk_student_drive` refuses a second row even if the check is bypassed.

Q LAZY versus EAGER, and what is a detached entity?

EAGER loads the related row with the entity, LAZY waits until touched. A detached entity has lost its transaction, as in a servlet; a lazy list on it throws `LazyInitializationException` . `merge` reattaches it.

Q What happens when an exception is thrown inside a transaction?

A runtime exception is a system exception: the container rolls the transaction back and the caller gets it wrapped in `EJBException`, so the servlet reads `getCause()`, as `withdraw` does. The audit row survives (`REQUIRES_NEW`). Reads use `SUPPORTS`, writes `REQUIRED`.

Q How does the MDB work, and can a placed student apply again?

`NotificationSender.send` queues a text message; the container calls `NotificationMDB.onMessage`, which logs it and saves a NOTIFICATION row. A placed student can apply again: no rule forbids it.